Università degli Studi di Udine

Dipartimento di Scienze Matematiche,
Informatiche e Fisiche

IBML - Internet of Things, Big Data, Machine Learning

Internet of Things
Progetto Pet Tracker

Autori

Andrea Gioia, 169484	169484@spes.uniud.it
Alessio Nicodemo, 166972	166972@spes.uniud.it
Mattia Nadal, 167372	167372@spes.uniud.it
Francesco Sabot, 167339	167339@spes.uniud.it

Abstract

Il progetto Pet-Tracker nasce con l'obiettivo di realizzare un sistema di localizzazione GPS in tempo reale per animali domestici di media taglia, in grado di trasmettere la posizione, lo stato della batteria ed il numero di passi verso un server remoto accessibile tramite internet, e di visualizzare tali dati attraverso un'applicazione dedicata.

Indice

1 Architettura del sistema

L'architettura del sistema Pet-Tracker segue il **Modello a 7 livelli dell'IoT World Forum (IoTWF)**, garantendo una transizione fluida dal dato fisico all'informazione per l'utente.

1.1 Mappatura sul modello IoTWF

- **Livello 1 (Physical Devices):** Rappresentato dalla board LilyGo e dai sensori (GPS, Accelerometro).
- **Livello 2 (Connectivity):** Gestito tramite il modem LTE e protocolli di trasporto sicuri.
- **Livello 3 (Edge Computing):** Elaborazione locale sul microcontrollore per il filtraggio delle stringhe NMEA e il calcolo dei passi.
- **Livello 4 (Data Accumulation):** Persistenza dei dati sul server PocketBase (database SQLite).
- **Livello 5 (Data Abstraction):** Esposizione dei dati tramite API REST strutturate in formato JSON.
- **Livello 6 (Application):** L'applicazione mobile dedicata che trasforma le coordinate in posizioni su mappa.
- **Livello 7 (Collaboration and Processes):** L'interazione del proprietario con il sistema per il monitoraggio dell'animale.

2 Hardware

2.1 LiLyGo T-SIM7670G-S3

Il dispositivo utilizza una board LiLyGo T-SIM7670G-S3 [**lilygo˙specs**] che si basa sul modulo **ESP32-S3** dual-core, che gestisce sia la logica applicativa che la comunicazione con il modem SIM7670G tramite interfaccia UART.

2.2 Antenna GPS e LTE

Il sistema utilizza un'antenna flessibile LTE [**LTE**] per la connettività dati e un'antenna ceramica attiva [**GPS**] per il ricevitore GNSS integrato, essenziale per la precisione della localizzazione outdoor.

2.3 Batteria e Power Management

Viene impiegata una batteria ricaricabile 3000mAh 30A Litio-Ion ad alta capacità. Il monitoraggio avviene tramite un partitore di tensione collegato al pin `ADC_BAT` (GPIO 4), permettendo di calcolare la percentuale di carica residua.

2.4 BMA400 CJMCU-400

L'accelerometro usato è un **BMA400** [**bma400**] integrato nella breadboard **CJMCU-400**, che viene interrogato per rilevare il movimento, consentendo l'implementazione di un contapassi utile al monitoraggio della salute dell'animale.

3 Firmware: Logica di funzionamento

Il firmware, sviluppato in ambiente Arduino, implementa una macchina a stati per l'acquisizione e l'invio:

1. **Inizializzazione:** Configurazione dei pin, avvio della seriale hardware per il modem e attivazione del motore GNSS tramite comandi AT [**sim7670˙at**] (`AT+CGNSSPWR=1`).
2. **Loop Principale:**
   - Lettura dei dati GNSS tramite il comando `AT+CGNSSINFO`.
   - Acquisizione dello stato batteria (`leggiBatteria()`).
   - Parsing dei dati per verificare la validità del fix GPS.
3. **Trasmissione:** Se il fix è valido, i dati vengono incapsulati in una stringa JSON e inviati tramite una richiesta `HTTPS POST` all'endpoint di PocketBase.

4 Back-end e Database

Il centro nevralgico della gestione dati è basato su **PocketBase** [**pocketbase**] , una soluzione open-source che integra un database embedded SQLite con un'interfaccia API REST.

4.1 Architettura di Hosting e Network Exposure

L'istanza di PocketBase è stata ospitata su una workstation locale e data la natura del sistema IoT, che richiede la raggiungibilità del backend da parte del dispositivo (tramite rete LTE) e dell'applicazione mobile (tramite rete dati/Wi-Fi), si è reso necessario esporre il servizio locale su rete pubblica. Il problema del *NAT Traversal* e dell'assenza di un indirizzo IP pubblico statico sulla rete locale è stato risolto mediante l'impiego di **ngrok** [**ngrok**]. Questa tecnologia di *tunneling* crea un tunnel sicuro (HTTPS) tra l'endpoint pubblico di ngrok e la porta locale su cui è in esecuzione il server. Tale URL funge da *Reverse Proxy*, reindirizzando il traffico HTTPS in ingresso verso l'istanza locale di PocketBase, garantendo così la persistenza dei dati e l'accessibilità dei servizi in tempo reale ovunque sia presente una connessione internet.

4.2 Sistema di Backup e Data Retention

Per garantire la resilienza dei dati e prevenire perdite accidentali dovute a guasti del server locale, è stato implementato un sistema di backup automatizzato. Il processo utilizza l'utility **rclone** [**rclone**], uno strumento a riga di comando per la gestione del cloud storage, configurato per eseguire la sincronizzazione della directory dei dati di PocketBase

(inclusi il database SQLite e gli eventuali asset) verso un bucket remoto su **Backblaze B2** [**backblaze**]. L'automazione è gestita tramite un *cron job* (o un'operazione pianificata) che esegue il backup in modo incrementale ogni primo giorno del mese. Questa strategia assicura che, nonostante l'hosting sia inizialmente locale, il sistema rispetti i criteri di persistenza a lungo termine.

4.3 Monitoraggio e Business Continuity

Per garantire l'affidabilità dell'intero ecosistema, è stato implementato un sistema di monitoraggio esterno tramite **UptimeRobot** [**uptimerobot**]. Dato che l'accessibilità del backend dipende dalla stabilità del tunnel ngrok e dall'operatività del server locale, il monitoraggio è configurato per interrogare l'endpoint HTTPS a intervalli regolari. In caso di mancata risposta (dovuta a un crash di ngrok, interruzione di rete o downtime del server), il sistema invia notifiche push e email immediate al team di sviluppo per interventi tecnici sulla disponibilità del servizio. Tale approccio permette di minimizzare il *Mean Time To Repair* (MTTR) e garantisce una maggiore robustezza della soluzione proposta.

4.4 Struttura dei dati

I dati ricevuti vengono normalizzati e memorizzati in una collezione denominata `positions`. La struttura del record prevede i seguenti campi:

- `timestamp`: riferimento temporale dell'acquisizione.
- `lat` / `lon`: coordinate geografiche acquisite dal modulo GNSS;
- `battery`: stato di carica della batteria (espresso in voltaggio);
- `battery_percent`: stato di carica della batteria (espresso in percentuale);
- `charging`: booleano che indica se la batteria è in corso di ricarica;
- `feet`: conteggio dei passi rilevato dall'accelerometro BMA400;

4.5 Interfaccia API e API Rules

L'accesso ai dati memorizzati nel backend avviene esclusivamente tramite un'interfaccia **RESTful API** fornita nativamente da PocketBase. Questo approccio permette di separare nettamente il livello di persistenza (Database) dal livello di fruizione (Applicazione Mobile), in linea con il Livello 5 (Data Abstraction) del modello IoTWF.

4.5.1 API Rules e Sicurezza

Per garantire l'integrità e la riservatezza dei dati, sono state configurate specifiche **API Rules**. A differenza di un database tradizionale, PocketBase utilizza un sistema di regole basato su espressioni booleane per filtrare le operazioni di:

- **List/View:** Definisce chi può leggere i record. Nel nostro caso, è stata impostata una regola che permette la visualizzazione solo agli utenti autenticati.
- **Create:** Permette l'inserimento di nuovi dati. Questa operazione è abilitata per il dispositivo LilyGo tramite un token di autenticazione o restrizioni sull'origine delle richieste.

- **Update/Delete:** Queste operazioni sono state disabilitate o ristrette ai soli amministratori del sistema per evitare la manomissione accidentale dello storico delle posizioni.

L'utilizzo delle API Rules sostituisce la necessità di scrivere codice di back-end personalizzato per la validazione, permettendo di gestire in modo granulare i permessi direttamente dal pannello di controllo del server.

5 Front-end e Applicazione Mobile

L'applicazione mobile, sviluppata in Flutter [**flutter**], è progettata per interagire con i dati tramite il PocketBase SDK, che semplifica l'interazione con l'endpoint HTTPS e la gestione dei token di autenticazione, garantendo una comunicazione asincrona e performante.

6 Test e Risultati

7 Conclusioni